

Sunrise Dental Clinic — Appointment & Patient Management System

CIS6003 Advanced Programming — WRIT1

Tasks B, C and D: Design Patterns & Architecture, Testing, and Version Control / CI-CD

Introduction

This document presents Tasks B, C and D of the Sunrise Dental Clinic coursework, continuing from the system design established in Task A. Task A modelled the problem with UML (use-case, class and sequence diagrams); this report explains how that design was realised as a working, distributed, cloud-backed application, how it was tested using test-driven development, and how it was version-controlled and delivered through a continuous-integration and deployment pipeline.

The system replaces Sunrise Dental Clinic's paper-based process — the source of double bookings, lost records, long waits and billing errors — with a role-based web application serving four actors: Patient, Receptionist, Dentist and Administrator. Throughout, the engineering approach was informed by patterns proven in a production education platform, adapted to this domain and stack (React, Spring Boot and Firebase).

Task B — Design Patterns and Architecture Development

B.1 A three-tier, distributed architecture

The system is deliberately built as a three-tier, distributed application so that each concern can evolve and scale independently. The presentation tier is a React single-page application; the business tier is a Spring Boot REST service written in Java; and the data tier is Google Firestore (a managed cloud database), with Firebase Authentication for identity and Firebase Cloud Functions for event-driven work. The three tiers run as separate, independently deployable processes and communicate over

HTTPS/JSON, which is what makes the application genuinely distributed rather than a single monolith.

Tier	Technology	Responsibility
Presentation	React 18 + TypeScript (Vite)	Role-aware UI: dashboards, booking wizard, charts, AI chat
Business	Spring Boot 3 (Java 17) REST API	Controllers → services → repositories; design patterns; security
Data	Firestore (prod) / in-memory (test) + Firebase Auth	Persistence behind a Repository abstraction; identity
Serverless	Firebase Cloud Functions	onCreate notification trigger; scheduled reminders

Table B.1 — The three-tier, distributed architecture.

Communication between the browser and the API is via REST web services; because the React client and the Spring Boot server are separate deployables that talk over HTTP, the presentation and business tiers can be hosted, scaled and released independently. This satisfies the requirement for a distributed application exposing web services.

B.2 Design patterns implemented

Seven Gang-of-Four and enterprise patterns were applied. Each was chosen to solve a specific problem in the domain rather than for its own sake; the table summarises them and the subsections evaluate the most significant.

Pattern	Where	Problem it solves
Singleton	AppointmentNumberGenerator	One thread-safe source of unique APT-yyyymmdd-#### numbers
Strategy	BillingStrategy,	Swappable pricing and the

	RevenueSplitStrategy	3-way revenue split
Factory Method	NotificationChannelFactory	Create the right Email/SMS channel by type
Observer	AppointmentEventPublisher	Fan out notify + audit when an appointment is created
Repository/DAO	*Repository (in-memory + Firestore)	Swap persistence; test offline (Dependency Inversion)
DTO (role-scoped)	AppointmentResponse / ... DetailResponse	Field-level confidentiality of the diagnosis
Facade / Service + MVC + DI	*Service, Spring MVC	Separate layers; controllers never touch data directly

Table B.2 — Design patterns and their purpose.

Strategy — billing and the three-way revenue split

Billing is expressed as a Strategy so the pricing rule can change without touching callers. A BillingStrategy computes the bill (consultation fee + treatment cost + service charge), and a separate RevenueSplitStrategy divides each bill three ways: the consultation fee to the dentist, the treatment cost 60% dentist / 40% clinic, and the service charge to the receptionist who handled it. Because both are interfaces, an alternative rule — a promotional discount, a different split — can be introduced as a new class with no change to the BillingService. This is the open/closed principle in practice and directly powers the Administrator's income reports.

Observer — notifications and audit without coupling

When an appointment is created the AppointmentEventPublisher notifies its observers. One observer sends the patient a confirmation (through a Factory-created channel), another writes an audit record. New reactions to the event — an SMS reminder, an analytics counter — are added by registering another observer, so the booking code never grows to know about them. This mirrors the event-driven

decoupling used in the reference platform and keeps the core booking transaction focused.

Repository/DAO — a swappable, testable data tier

Every aggregate is reached through a Repository interface with two implementations: an in-memory adapter used by the tests and offline demos, and a Firestore adapter for production. Because the services depend on the interface, not on Firestore, the entire business tier can be unit-tested in milliseconds with no cloud credentials, while the same code runs against the real database in production. This is the Dependency Inversion Principle and it is the single most valuable structural decision in the codebase.

DTO with role-scoped fields — a fail-safe confidentiality boundary

A patient's clinical diagnosis must be visible only to the treating dentist and the patient — never to reception or admin. Rather than filter fields at runtime (which fails open if a check is forgotten), the design uses two separate response types:

AppointmentResponse has no diagnosis field at all, while

AppointmentDetailResponse adds it and is returned only when an access check confirms the caller is the dentist-owner or the patient. Because the field does not exist on the class served to other roles, it cannot leak by accident. This static-exclusion approach was adopted directly from the reference platform's serializer design.

B.3 Database design and advanced data features

The production data tier is Firestore, a document database organised as collections: patients, dentists, dentistSessions, slots, treatments, appointments, bills, notifications, auditEvents and a counters document that backs the appointment-number sequence.

Three features implement business rules at the data tier:

- Atomic booking (transactions): a slot is booked inside a Firestore transaction that re-checks the slot is still OPEN before flipping it to BOOKED, making the double-booking the clinic suffered on paper impossible.
- Cloud Function triggers: an onCreate trigger on the appointments collection fires the confirmation notification, and a scheduled function sends a 24-hour reminder — the cloud-native equivalent of database triggers implementing business rules.

- Security rules: Firestore security rules enforce the access matrix at the database itself (defence in depth), so even a mis-routed request cannot read another patient's record or a diagnosis it should not see.

B.4 Security, RBAC and field-level confidentiality

Access control operates at three levels. Role-based method security (@PreAuthorize) guards each endpoint by role; object-level checks confirm the caller owns the record; and the DTO boundary above hides confidential fields. The authentication convention is explicit and consistent: 401 for an unauthenticated request, 403 for an authenticated request with the wrong role, and 404 when a caller asks for a record outside their scope (so the API never reveals that another patient's record exists). An account-lockout service locks an identity for 24 hours after five failed logins, tracking failedAttempts and a lockedUntil timestamp — the 'lock account after failed attempts' use case from Task A.

B.5 Web services (REST API)

The business tier exposes a REST API consumed by the React client. The principal resources:

Method & path	Role	Purpose
POST /api/appointments	Patient/Recep/Admin	Book an appointment (atomic slot claim)
GET /api/appointments/{no}	owner/dentist	Retrieve — clinical view only for dentist/patient
POST /api/appointments/{no}/complete	Dentist	Record diagnosis, mark completed
POST /api/appointments/{no}/bill	Recep/Admin	Generate bill + revenue split
GET /api/availability?dentistId&date	any signed-in	Open slots for a dentist on a date
POST /api/sessions	Recep/Admin	Publish a dentist's

		availability window
POST /api/patients	any signed-in	Register a patient
/api/auth/*	varies	Login-attempt lockout state

Table B.3 — Core REST endpoints.

B.6 Advanced and innovative features

- AI symptom triage: a patient describes a dental problem and the assistant suggests a likely issue, urgency and treatment. The design borrows the reference platform's AI safety pattern — a grounding check before the model is called and an idempotent disclaimer added after — so the assistant never diagnoses and always advises seeing a dentist. A deterministic rules-based fallback answers when the model is unavailable.
- Runtime-configurable AI model: the Administrator can change the AI provider/model (e.g. gemini-1.5-flash → gemini-1.5-pro) and temperature from the dashboard; the change takes effect immediately via a cached settings record, with the API key kept server-side.
- Decision-support reports: income by day/week/month with the three-way split, daily footfall by dentist and by receptionist, and per-dentist earnings — with CSV export.
- Email/SMS confirmations (Cloud Function trigger + Observer), PDF/printable receipts, and an audit log of every significant action.

B.7 Evaluation

The architecture meets the brief's most demanding criteria: a distributed application with web services, a real cloud database with trigger-driven business rules, a three-tier separation, and multiple, well-justified design patterns. The chief trade-off is that a document database (Firestore) sacrifices relational joins and SQL stored procedures; this was accepted because Firestore's transactions, security rules and Cloud-Function triggers cover the same business-rule needs while giving managed scaling and a generous free tier. The Repository abstraction hedges this decision — were a relational database preferred later, only the data-tier adapter would change.

Task C — Testing

C.1 Rationale and test-driven development

Testing followed a test-driven development (TDD) cycle: for each behaviour a failing test was written first (red), the minimum code to satisfy it was implemented (green), and the design was then refined (refactor). Writing the test first forced the interface to be specified before the implementation — for example, the billing test fixed the shape of the revenue split before `BillingService` existed. Tests run against the in-memory Repository, so the whole suite executes in milliseconds with no cloud credentials, which keeps the red-green loop fast and makes the suite safe to run on every push in CI.

C.2 Test plan

The plan is tiered, mirroring the layers of the application. Each layer has at least one positive case and the appropriate negative cases (401/403/400).

Layer	What it verifies	Example classes
Pattern unit	Each pattern in isolation	<code>AppointmentNumberGeneratorTest</code> , <code>StandardBillingStrategyTest</code> , , <code>DefaultRevenueSplitStrategyTest</code> , <code>NotificationChannelFactoryTest</code>
Service	Business rules across collaborators	<code>AppointmentServiceTest</code> , <code>BillingServiceTest</code>
Security	Lockout state machine	<code>LoginAttemptServiceTest</code>
Web (MockMvc)	Endpoints, status codes, auth	<code>AppointmentApiTest</code> , <code>AuthApiTest</code>

Table C.1 — Tiered test plan.

C.3 Test data

Test data is built with small factory helpers so each test starts from a valid object graph. Representative data used across the suite:

Input	Value(s)	Used to verify
Treatment cost	Rs 5,000 (Filling)	Revenue split: Rs 3,000 dentist / Rs 2,000 clinic
Consultation fee	Rs 1,500	Whole fee attributed to the dentist
Service charge	Rs 200	Attributed to the receptionist
Failed logins	5 consecutive	Account locks; lockedUntil = now + 24h
Slot double-book	two bookings, one slot	Second booking rejected
Wrong role	reception reads diagnosis	Diagnosis absent from response

Table C.2 — Representative test data.

C.4 Test automation

The suite is automated in the continuous-integration pipeline: the GitHub Actions workflow runs 'mvn test' on every push and pull request, and a pull request cannot be merged to the protected branch unless the tests pass. This turns the test plan into an enforced gate rather than a manual step.

C.5 Results

The full suite of 25 tests across 9 test classes executes with 0 failures. The results below are from a live run of 'mvn test' on the develop branch.

Metric	Result
Test classes	9
Tests run	25
Failures / errors	0

Outcome	BUILD SUCCESS

Table C.3 — Live test-run summary (mvn test).

[Insert screenshot: terminal output of 'mvn test' showing BUILD SUCCESS, and the GitHub Actions run showing the test job green.]

C.6 Requirement traceability

Every acceptance requirement traces to a test, so it is clear how the design is verified.

Req.	Requirement	Test	Status
R1	Unique appointment number	AppointmentNumberGeneratorTest	Automated
R2	No double-booking of a slot	AppointmentServiceTest	Automated
R3	Bill = consult + treatment + service	StandardBillingStrategyTest	Automated
R4	3-way revenue split correct	DefaultRevenueSplitStrategyTest	Automated
R5	Diagnosis hidden from wrong role	AppointmentApiTest	Automated
R6	Correct notification channel	NotificationChannelFactoryTest	Automated
R7	Lock account after 5 failures	LoginAttemptServiceTest	Automated
R8	Bill only after completion	BillingServiceTest	Automated
R9	401/403 on protected endpoints	AuthApiTest	Automated

Table C.4 — Requirement → test traceability matrix.

C.7 Evaluation and lessons learned

Writing tests first paid off most where the rules were subtle — the revenue split and the lockout state machine were correct on the first implementation because their tests pinned the expected numbers and transitions in advance. The main lesson was the value of the in-memory Repository: by inverting the data-tier dependency, the suite stays fast and hermetic, which in turn made continuous integration cheap enough to run on every commit. A future extension would add end-to-end UI tests (Playwright) against the React client to cover the presentation tier, complementing the current unit and web-slice coverage.

Task D — Version Control, CI/CD and Deployment

D.1 Repository and access

The project is developed in a public GitHub repository (github.com/ShakyaRanasinghe/sunrise-dental-clinic), so the history and every version are openly accessible. Branch protection on the main and develop branches restricts direct pushes and requires pull-request review and green CI before a change lands — the 'accessibility restrictions' the brief asks to be demonstrated.

D.2 Branching strategy and versioning

A GitFlow-style model is used: short-lived feature/* and chore/* branches are opened from develop, reviewed by pull request, and merged into develop; develop is promoted to qa for release-candidate testing and to main for release. Versions follow semantic versioning and are recorded in a single source of truth, version.json, at the repository root; a release is tagged (for example v0.2.0), and release candidates carry a -qa-rc-N suffix so it is obvious at a glance whether a tag is new code or a re-test. This convention was adopted from the reference platform's release process.

Branch	Purpose	Deploys to
feature/* · chore/*	One change each; PR-reviewed	— (CI only)
develop	Integration trunk, always	dev channel

	green	
qa	Release-candidate validation	qa channel
main	Released code	prod (live) channel

Table D.1 — Branch-to-environment mapping.

D.3 Version-control techniques

- Conventional Commits (feat, fix, chore, docs, test with a scope) give a readable, machine-parseable history and drive the changelog.
- Every change is a pull request linked to an issue number, so the history explains not just what changed but why.
- A CHANGELOG records each version's changes as its own atomic commit, separate from the code and version-bump commits, keeping history clean and reviewable.
- Feature isolation via git worktrees let the frontend be built on its own branch without disturbing concurrent backend work — merged cleanly through a pull request.

D.4 Continuous integration and deployment (workflow)

Four GitHub Actions workflows implement the pipeline:

Workflow	Trigger	Does
ci.yml	every push / PR	Build + run the full test suite; blocks merge on failure
deploy-dev.yml	push to develop	Build and deploy to the dev channel
deploy-qa.yml	qa branch / rc tag	Full suite + deploy to the qa channel
deploy-prod.yml	release / v* tag	Smoke test + deploy to the live channel

Table D.2 — CI/CD workflows.

The three environments (dev, qa, prod) are implemented as Firebase Hosting channels on one project, giving each a distinct URL at no cost. Production deployment is gated by a GitHub Environment protection rule requiring manual approval, and a rollback is an instant channel re-point — a lightweight blue-green scheme.

D.5 Deployment demonstration

[Insert screenshots: (1) the GitHub Actions runs list showing ci / deploy workflows green; (2) the GitHub Environments page showing dev/qa/prod with the prod approval rule; (3) the deployed application's live URL rendering the login page; (4) the commit graph / tags showing several versions.]

The latest version on main is the one deployed to the live channel; each promotion from develop → qa → main corresponds to a tagged version, so the deployed application is always traceable to an exact commit.

D.6 Evaluation

The chosen workflow is proportionate to a small team: it enforces quality (tests gate every merge) and provides safe, reversible releases (approval-gated prod, channel rollback) without the overhead of a full blue-green infrastructure. It scales — additional environments or a container deploy could be added as new workflows — and it makes the version history a reliable, auditable record of how the software evolved.

Conclusion

Tasks B–D took the Task A design to a working, distributed system. Task B realised a three-tier architecture with web services, a real cloud database with trigger-based business rules, and seven design patterns each solving a concrete problem — most notably the fail-safe diagnosis-confidentiality boundary and the Strategy-based revenue split. Task C verified the system with a fast, hermetic, test-driven suite, automated in CI and traced to requirements. Task D delivered it through a GitFlow, semantically-versioned repository and a dev/qa/prod CI-CD pipeline with gated, reversible releases. Together they demonstrate not only that the system works, but that it was engineered, tested and delivered to a professional standard.

References

- Beck, K. (2003) Test-Driven Development: By Example. Boston: Addison-Wesley.
- Evans, E. (2004) Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley.
- Fowler, M. (2003) Patterns of Enterprise Application Architecture. Boston: Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley.
- Google (2024) Cloud Firestore Documentation. Available at: <https://firebase.google.com/docs/firestore> (Accessed: 20 July 2026).
- Humble, J. and Farley, D. (2010) Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston: Addison-Wesley.
- Martin, R.C. (2017) Clean Architecture: A Craftsman's Guide to Software Structure and Design. Upper Saddle River, NJ: Prentice Hall.
- Meta Open Source (2024) React Documentation. Available at: <https://react.dev> (Accessed: 20 July 2026).
- OWASP Foundation (2021) OWASP Top Ten. Available at: <https://owasp.org/www-project-top-ten/> (Accessed: 20 July 2026).
- VMware (2024) Spring Boot Reference Documentation. Available at: <https://docs.spring.io/spring-boot/> (Accessed: 20 July 2026).